

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Алгоритмы и структуры данных»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Сортировка распределением (карманная сортировка)
в кольцевой очереди на базе связного списка»

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250

(подпись)

Проверил:

Ерофеев Сергей Анатольевич

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ВВЕДЕНИЕ.....	3
1. АЛГОРИТМ СОРТИРОВКИ.....	4
2. ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ПЕРЕМЕННЫХ.....	5
3. БЛОК-СХЕМА АЛГОРИТМА.....	7
4. ТЕСТИРОВАНИЕ АЛГОРИТМА.....	7
5. ИСХОДНЫЙ КОД АЛГОРИТМА.....	8
ЗАКЛЮЧЕНИЕ.....	12
ПРИЛОЖЕНИЕ А.....	13

ВВЕДЕНИЕ

Цель работы — разработать программу сортировки распределением (карманной сортировки) последовательности вещественных чисел, которая будет храниться в кольцевой очереди, построенной на базе связанного списка, определить сложность алгоритма путём подсчёта количества элементарных операций, выполняемых в программе.

1. АЛГОРИТМ СОРТИРОВКИ

Программа получает на вход название файла, в котором хранятся исходные числа для сортировки. Числа читаются из файла и записываются в кольцевую очередь.

Далее происходит сортировка очереди по следующему алгоритму:

1) Создаются карманы (тоже отдельные очереди), количество карманов равно количеству элементов в очереди;

2) Далее элементы распределяются по карманам. Для начала определяются максимальный (max) и минимальный (min) элементы очереди. Индекс нужного кармана для каждого элемента определяется по формуле: $\frac{(x - min) \cdot (len(queue) - 1)}{max - min}$.

3) Каждый заполненный карман сортируется с помощью сортировки вставками;

4) Содержимое карманов последовательно сливается в одну кольцевую очередь.

В результате получаем отсортированную очередь и печатаем ее в стандартный поток вывода.

2. ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ПЕРЕМЕННЫХ

Ниже представлена таблица с описанием использованных переменных.

Таблица 1 — Используемые переменные

Название	Тип	Границы типа	Назначение
value	float	от $-3,4E+38$ до $3,4E+38$	Значение узла очереди
next	struct Node*	-	Указатель на следующий узел очереди
head	struct Node*	-	Указатель на начальный узел очереди
tail	struct Node*	-	Указатель на конечный узел очереди
size	size_t	от 0 до $2^{64} - 1$	Размер очереди
node	struct Node*	-	Указатель на узел очереди
oldHead	struct Node*	-	Указатель на удаляемый узел
cur	struct Node*	-	Указатель на текущий узел
sortedHead	struct Node*	-	Указатель на начальный узел отсортированной части очереди
sortedTail	struct Node*	-	Указатель на конечный узел отсортированной части очереди
unsorted	struct Node*	-	Указатель на неотсортированный узел
min	float	от $-3,4E+38$ до $3,4E+38$	Минимальный элемент очереди
max	float	от $-3,4E+38$ до $3,4E+38$	Максимальный элемент очереди
numBuckets	int	от 0 до 2147483647	Количество карманов

			для сортировки
buckets	struct RingQueue*	-	Указатель на выделенную память для карманов
ind	int	от 0 до 2147483647	Индекс кармана для элемента очереди
val	float	от -3,4E+38 до 3,4E+38	Значение удаленного элемента из очереди
file	FILE*	-	Указатель на файловый поток для чтения
tmp	float	от -3,4E+38 до 3,4E+38	Переменная для хранения прочитанного числа из файла
argc	int	от 1 до 2147483647	Хранит количество аргументов при запуске программы
argv[]	char**	адрес памяти	Массив переданных аргументов
filename	char*	адрес памяти	Ссылается на имя файла
q	struct RingQueue*	-	Указатель на кольцевую очередь
i	int	от 0 до 2147483647	Переменная для итерации циклов

3. БЛОК-СХЕМА АЛГОРИТМА

Блок-схема алгоритма представлена в Приложении А.

4. ТЕСТИРОВАНИЕ АЛГОРИТМА

```
ЛР2 > ./lab2 data.txt
Исходные данные из файла:
[ 3.000000 -> 43.000000 -> 4325.000000 -> 1.000000 -> 46.000000 -> 12.440000 -> -123.339996 ->
 21947840.000000 -> 300.000000 -> 1.000000 -> -200.000000 ] (size=11)
Отсортированные данные:
[ -200.000000 -> -123.339996 -> 1.000000 -> 1.000000 -> 3.000000 -> 12.440000 -> 43.000000 ->
 46.000000 -> 300.000000 -> 4325.000000 -> 21947840.000000 ] (size=11)
ЛР2 > ./lab2 data.txt
Исходные данные из файла:
[ пусто ]
Отсортированные данные:
[ пусто ]
```

Рисунок 1 — Результат успешной работы алгоритма

```
ЛР2 > ./lab2
Использование: ./lab2 <имя_файла>
```

Рисунок 2 — Результат работы алгоритма при неуказанном входном файле

5. ИСХОДНЫЙ КОД АЛГОРИТМА

Программа написана на языке C, для компиляции использовался компилятор gcc.

Исходный код Makefile:

```
1 | CC = gcc
2 | CFLAGS = -O3 -Wall -Wextra -Werror -fanalyzer
3 | TARGET = lab2
4 |
5 | all: $(TARGET)
6 |
7 | $(TARGET): lab2.c
8 |     $(CC) $(CFLAGS) $(TARGET).c -o $(TARGET)
9 |
10 | clean:
11 |     rm -f $(TARGET)
```

Исходный код lab2.c:

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include "stddef.h"
4 | #include "stdbool.h"
5 |
6 | typedef struct Node {
7 |     float value;
8 |     struct Node *next;
9 | } Node;
10 |
11 | typedef struct RingQueue {
12 |     struct Node *head;
13 |     struct Node *tail;
14 |     size_t size;
15 | } RingQueue;
16 |
17 | void init(RingQueue *q) {
18 |     q->head = q->tail = NULL;           // (6)
19 |     q->size = 0;                         // (3)
20 | }
21 | // Итоговая сложность init: 6 + 3 = 9
22 |
23 | void enqueue(RingQueue *q, float value) {
24 |     Node *node = malloc(sizeof(Node));   // (3)
25 |     if (!node) {                         // (2)
26 |         fprintf(stderr, "Ошибка: не удалось выделить память для узла кольцевой очереди.
27 |         \n");
28 |         exit(1);
29 |     }
30 |     node->value = value;                  // (3)
31 |     if (q->size == 0) {                   // (3)
32 |         node->next = node;                 // (3)
33 |         q->head = node;                   // (3)
34 |         q->tail = node;                   // (3)
35 |     } else {
36 |         node->next = q->head;               // (5)
37 |         q->tail->next = node;               // (5)
38 |         q->tail = node;                   // (3)
39 |     }
40 |
41 |     q->size++;                             // (4)
42 | }
43 | // Итоговая сложность enqueue (худший случай - ветка else): 3 + 2 + 3 + 3 + 5 + 5 + 3 +
44 |
```



```

45 | bool dequeue(RingQueue *q, float *out) {
46 |     if (q->size == 0) return false; // (4)
47 |     Node *oldHead = q->head; // (3)
48 |     *out = oldHead->value; // (4)
49 |
50 |     if (q->size == 1) { // (3)
51 |         q->head = q->tail = NULL; // (6)
52 |     } else {
53 |         q->head = oldHead->next; // (5)
54 |         q->tail->next = q->head; // (7)
55 |     }
56 |
57 |     free(oldHead); // (1)
58 |     q->size--; // (4)
59 |     return true; // (1)
60 | }
61 | // Итоговая сложность dequeue (худший случай - ветка else): 4 + 3 + 4 + 3 + 5 + 7 + 1 +
4 + 1 = 32
62 |
63 | void printQ(RingQueue *q) {
64 |     if (q->size == 0) { // (3)
65 |         printf("[ пусто ]\n");
66 |         return;
67 |     }
68 |
69 |     Node *cur = q->head; // (3)
70 |     printf("[ "); // (1)
71 |     for (size_t i = 0; i < q->size; i++) { // (1) + (3)*(n+1) + (2)*(n) =
5n + 4
72 |         printf("%f ", cur->value); // (3)
73 |         if (i + 1 < q->size) printf("-> "); // (5)
74 |         cur = cur->next; // (3)
75 |     } // Тело цикла: 3 + 5 + 3 = 11. Итого цикл: 5n + 4 + 11n = 16n + 4
76 |     printf(" ] (size=%zu)\n", q->size); // (3)
77 | }
78 | // Итоговая сложность printQ: 3 + 3 + 1 + (16n + 4) + 3 = 16n + 14
79 |
80 | void insertionSort(RingQueue *q) {
81 |     size_t n = q->size; // (3)
82 |     if (n <= 1) return; // (2)
83 |
84 |     Node *sortedHead = q->head; // (3)
85 |     Node *sortedTail = q->head; // (3)
86 |     Node *unsorted = q->head->next; // (5)
87 |
88 |     sortedHead->next = sortedHead; // (3)
89 |
90 |     for (size_t i = 1; i < n; i++) { // (1) + (1)*(n) + (2)*(n-1) =
3n - 1
91 |         Node *cur = unsorted; // (1)
92 |         unsorted = cur->next; // (3)
93 |
94 |         if (cur->value <= sortedHead->value) { // (5)
95 |             cur->next = sortedHead; // (3)
96 |             sortedTail->next = cur; // (3)
97 |             sortedHead = cur; // (1)
98 |         } else if (cur->value >= sortedTail->value) { // (5)
99 |             cur->next = sortedHead; // (3)
100 |             sortedTail->next = cur; // (3)
101 |             sortedTail = cur; // (1)
102 |         } else {
103 |             Node *prev = sortedHead; // (1)
104 |             while (prev->next != sortedHead && prev->next->value <= cur->value) { //
(11)
105 |                 prev = prev->next; // (3)
106 |             } // Худший случай while (проход i элементов): 11*(i+1) + 3*i = 14i + 11
107 |             cur->next = prev->next; // (5)
108 |             prev->next = cur; // (3)
109 |         } // Худший случай ветки else: 1 + (14i + 11) + 5 + 3 = 14i + 20
110 |     } // Тело цикла (худший случай): 1 + 3 + 5 + 5 + (14i + 20) = 14i + 34
111 |     // Сумма от i=1 до n-1 для (14i + 34) = 14*(n(n-1)/2) + 34(n-1) = 7n^2 + 27n - 34

```

```

112 |         // Весь цикл:  $(3n - 1) + (7n^2 + 27n - 34) = 7n^2 + 30n - 35$ 
113 |         q->head = sortedHead; // (3)
114 |         q->tail = sortedTail; // (3)
115 |     }
116 |     // Итоговая сложность insertionSort:  $3 + 2 + 3 + 3 + 5 + 3 + (7n^2 + 30n - 35) + 3 + 3$ 
117 |     =  $7n^2 + 30n - 10$ 
118 |     void bucketSort(RingQueue *q) {
119 |         size_t n = q->size; // (3)
120 |         if (n <= 1) return; // (2)
121 |
122 |         float min = q->head->value, max = q->head->value; // (10)
123 |         Node *cur = q->head->next; // (5)
124 |         for (size_t i = 1; i < n; i++) { //  $(1) + (1)*(n) + (2)*(n-1) =$ 
125 |             if (cur->value < min) min = cur->value; // (6)
126 |             if (cur->value > max) max = cur->value; // (6)
127 |             cur = cur->next; // (3)
128 |         } // Цикл 1:  $(3n - 1) + 15(n-1) = 18n - 16$ 
129 |
130 |         if (min == max) return; // (2)
131 |
132 |         int numBuckets = (int)n; // (2)
133 |         RingQueue *buckets = malloc(numBuckets * sizeof(RingQueue)); // (4)
134 |         if (!buckets) { // (2)
135 |             fprintf(stderr, "Ошибка: не удалось выделить память для корзины\n"); exit(1);
136 |         }
137 |         for (int i = 0; i < numBuckets; i++) //  $(1) + (n+1) + 2n = 3n + 2$ 
138 |             init(&buckets[i]); // (12)
139 |         // Цикл 2:  $(3n + 2) + 12n = 15n + 2$ 
140 |
141 |         while (q->size != 0) { //  $(3) * (n+1) = 3n + 3$ 
142 |             float val; // (0)
143 |             dequeue(q, &val); // (34)
144 |             int ind = (int)((val - min) * (numBuckets - 1) / (max - min)); // (7)
145 |             enqueue(&buckets[ind], val); // (31)
146 |         } // Цикл 3:  $3n + 3 + n*(34 + 7 + 31) = 75n + 3$ 
147 |
148 |         for (int i = 0; i < numBuckets; i++) { //  $(1) + (n+1) + 2n = 3n + 2$ 
149 |             if (buckets[i].size == 0) continue; // (4)
150 |             insertionSort(&buckets[i]); //  $(3) + \text{insertionSort}(7k^2 +$ 
151 |             float val; // (0)
152 |             while (buckets[i].size != 0) { //  $(3) * (k+1)$ 
153 |                 if (dequeue(&buckets[i], &val)) { // (37)
154 |                     enqueue(q, val); // (29)
155 |                 }
156 |             } // Внутренний while для k элементов:  $3(k+1) + k*(37 + 29) = 69k + 3$ 
157 |         } // Худший случай цикла 4 (все n элементов в одной корзине, остальные n-1 пустые):
158 |         // Пустые корзины:  $(n-1) * 4 = 4n - 4$ 
159 |         // Полная корзина:  $3 \text{ (if)} + (7n^2 + 30n - 7) + (69n + 3) = 7n^2 + 99n - 1$ 
160 |         // Весь цикл 4:  $(3n + 2) + (4n - 4) + (7n^2 + 99n - 1) = 7n^2 + 106n - 3$ 
161 |
162 |         free(buckets); // (1)
163 |     }
164 |     // Итоговая сложность bucketSort:
165 |     //  $3 + 2 + 10 + 5 + (18n - 16) + 2 + 2 + 4 + 2 + (15n + 2) + (75n + 3) + (7n^2 + 106n -$ 
166 |     3) + 1 =  $7n^2 + 214n + 17$ 
167 |     void readFromFile(char *filename, RingQueue *q) {
168 |         FILE *file = fopen(filename, "r"); // (2)
169 |         if (!file) { // (2)
170 |             fprintf(stderr, "Ошибка: не удалось открыть файл %s\n", filename);
171 |             exit(1);
172 |         }
173 |
174 |         float tmp; // (0)
175 |         while (fscanf(file, "%f", &tmp) == 1) { //  $(3) * (n+1) = 3n + 3$ 
176 |             enqueue(q, tmp); // (29)
177 |         } // Цикл:  $3n + 3 + 29n = 32n + 3$ 
178 |

```

```

179 |         fclose(file);                                // (1)
180 |     }
181 |     // Итоговая сложность readFromFile:  $2 + 2 + (32n + 3) + 1 = 32n + 8$ 
182 |
183 |     int main(int argc, char **argv) {
184 |         if (argc == 2) {                                // (1)
185 |             char *filename = argv[1];                    // (2)
186 |             RingQueue q;                                  // (0)
187 |             init(&q);                                     // (11)
188 |             readFromFile(filename, &q);                  // (32n + 10)
189 |
190 |             printf("Исходные данные из файла:\n");       // (1)
191 |             printQ(&q);                                   // (16n + 16)
192 |             bucketSort(&q);                               // (7n^2 + 214n + 19)
193 |             printf("Отсортированные данные:\n");        // (1)
194 |             printQ(&q);                                   // (16n + 16)
195 |
196 |         } else {
197 |             fprintf(stderr, "Использование: %s <имя_файла>\n", argv[0]);
198 |         }
199 |
200 |         return 0;                                        // (1)
201 |     }
202 |     // Итоговая сложность main (и всей программы в худшем случае):
203 |     //  $1 + 2 + 11 + (32n + 10) + 1 + (16n + 16) + (7n^2 + 214n + 19) + 1 + (16n + 16) + 1 =$ 
7n^2 + 278n + 78

```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была разработана программа на языке C для сортировки последовательности чисел с помощью алгоритма распределения (карманная сортировка), использующего кольцевую очередь, построенную на базе связного списка. Для решения задачи также был реализован алгоритм сортировки вставками. Числа считывались из файла и хранились в кольцевой очереди.

Программа разрабатывалась в среде VS Code, компилировалась с помощью gcc и запускалась в ОС Arch Linux.

В конце работы была оценена сложность алгоритма путём подсчёта всех элементарных операций, выполняющихся в программе. Тестирование алгоритма показало его корректную работу при различных входных данных.

Выполнение работы позволило закрепить навыки разработки алгоритмов сортировки, а также углубить знания в области структур данных.

ПРИЛОЖЕНИЕ А



